

8086 - Organização da memória

O processador **Intel 8086 gerencia a memória através de um sistema chamado segmentação de memória.**

Na época de seu lançamento, a Intel enfrentou um desafio de engenharia: o 8086 possuía registradores internos de **16 bits**, mas os projetistas queriam que ele acessasse até **1 MB de memória física** (o que exige um barramento de endereço de **20 bits**, já que $2^{20} = 1.048.576$ bytes). A solução foi dividir a memória em blocos e combinar dois registradores diferentes para achar um endereço real.

Abaixo está o funcionamento detalhado desse gerenciamento:

1. O Conceito de Segmentação

Em vez de enxergar a memória como um bloco único e plano de 1 MB, o 8086 a divide logicamente em **segmentos de até 64 KB**.

- Cada segmento de 64 KB representa o limite máximo que um registrador comum de 16 bits consegue indexar ($2^{16} = 65.536$ bytes).
- O processador consegue manter **4 segmentos ativos simultaneamente**, utilizando registradores específicos para cada função:

Registrador de Segmento	Nome	Função
CS (<i>Code Segment</i>)	Segmento de Código	Onde ficam as instruções do programa.
DS (<i>Data Segment</i>)	Segmento de Dados	Onde ficam as variáveis e dados globais.
SS (<i>Stack Segment</i>)	Segmento de Pilha	Controla a pilha do sistema (chamadas de função e variáveis locais).
ES (<i>Extra Segment</i>)	Segmento Extra	Um segmento de dados adicional, muito usado para cópia de strings.

2. O Cálculo do Endereço Físico (20 bits)

Para encontrar um dado na memória real, o 8086 adota um esquema de **Endereço Lógico**, representado no formato **Segmento:Offset** (Segmento:Deslocamento).

O circuito interno da CPU (na Unidade de Interface de Barramento - BIU) calcula o **Endereço Físico de 20 bits** usando a seguinte regra matemática:

1. Pega o valor de 16 bits do **Registrador de Segmento**.

2. Desloca esse valor **4 bits para a esquerda** (o que equivale a multiplicar por 16 em decimal, ou adicionar um zero 0 à direita em hexadecimal).
3. Soma o valor de 16 bits do **Offset** (deslocamento).

Exemplo Prático:

Se o registrador de código **CS** contém **1234H** e o ponteiro de instrução **IP** (o offset) contém **0012H**, o cálculo é:

12340H	(Segmento deslocado / multiplicado por 16)
+ 0012H	(Offset)

12352H	<- Este é o Endereço Físico de 20 bits enviado ao barramento

3. Organização Física: Bancos de Memória

Fisicamente, os 1 MB de memória não estão dispostos em uma única trilha. O 8086 divide a memória em dois bancos físicos de 512 KB para acelerar o acesso a dados de 16 bits:

- **Banco Par (*Even Bank*)**: Contém os bytes com endereços pares (0, 2, 4...). É conectado às linhas de dados baixas (D₀ a D₇).
- **Banco Ímpar (*Odd Bank*)**: Contém os bytes com endereços ímpares (1, 3, 5...). É conectado às linhas de dados altas (D₈ a D₁₅).

Usando os sinais de controle do processador (A0 e BHE), o 8086 consegue ler ou gravar um byte isolado (em qualquer banco) ou uma **palavra de 16 bits inteira em um único ciclo de clock**, desde que essa palavra comece em um endereço par. Se uma palavra de 16 bits começar em um endereço ímpar, o processador precisará de dois ciclos de memória para acessá-la por completo.

Vantagens e Desvantagens desse modelo

- **Vantagem (Relocalização)**: Como o programa usa apenas referências de *offset* (relativas), um código pode ser movido para qualquer lugar da memória física apenas mudando o valor inicial nos registradores de segmento.
- **Desvantagem (A "Ginástica" dos 64K)**: Se o seu programa ou conjunto de dados passasse de 64 KB, o programador precisava alterar manualmente os registradores de segmento durante a execução, gerando códigos mais complexos e lentos (o que originou os famosos modelos de memória do DOS: *Tiny, Small, Compact, Large, Huge*).

Como os ponteiros Near e Far funcionam no 8086?

Os ponteiros **Near** e **Far** surgiram no 8086 justamente para lidar com as limitações e a estrutura do sistema de segmentação de memória (aquela divisão em blocos de até 64 KB).

A diferença fundamental entre eles está em **quantas informações** o ponteiro precisa guardar para encontrar um dado ou instrução na memória.

Veja como cada um funciona na prática:

1. Ponteiro Near (Ponteiro Próximo)

Um ponteiro **Near** possui apenas **16 bits** de tamanho. Ele armazena exclusivamente o **Offset** (deslocamento).

- **Como ele funciona:** Ele assume que o segmento atual (seja de código com **CS** ou de dados com **DS**) já está apontando para o lugar certo. O processador apenas pega esses 16 bits e os usa como o deslocamento dentro do segmento atual.
 - **Limitação:** Ele só consegue apontar para endereços que estejam dentro do mesmo segmento de 64 KB onde o código está rodando. Ele não consegue pular para fora dele.
 - **Vantagem:** É muito **rápido** e ocupa menos memória RAM (apenas 2 bytes). No assembly do 8086, uma instrução de pulo do tipo **JMP near** altera apenas o registrador **IP** (Instruction Pointer).
-

2. Ponteiro Far (Ponteiro Distante)

Um ponteiro **Far** possui **32 bits** de tamanho (4 bytes). Ele armazena a informação completa do endereço lógico: **Segmento (16 bits) + Offset (16 bits)**.

- **Como ele funciona:** Ele não depende dos registradores de segmento atuais (**CS** ou **DS**). Quando o processador lê um ponteiro Far, ele carrega os primeiros 16 bits diretamente no registrador de segmento correspondente e os outros 16 bits no registrador de offset.
 - **Alcance:** Ele consegue apontar para **qualquer lugar dentro de toda a memória de 1 MB** do 8086, pois ele redefine o segmento antes de acessar o dado.
 - **Desvantagem:** É **mais lento** e pesado. O processador precisa atualizar dois registradores internos em vez de um. No assembly, um **CALL far** (chamada de função distante) precisa salvar tanto o **CS** quanto o **IP** antigos na pilha antes de pular para a nova função.
-

Resumo das Diferenças

Característica	Ponteiro Near (Próximo)	Ponteiro Far (Distante)
Tamanho em RAM	2 bytes (16 bits)	4 bytes (32 bits)
Conteúdo	Apenas o <i>Offset</i>	<i>Segmento</i> + <i>Offset</i>
Alcance	Dentro do mesmo segmento (até 64 KB)	Qualquer lugar da memória (até 1 MB)
Desempenho	Mais rápido (muda 1 registrador)	Mais lento (muda 2 registradores)

O Impacto na Linguagem C (Modelos de Memória do DOS)

Na época do MS-DOS, compiladores C (como o Turbo C) usavam esses ponteiros para definir os **Modelos de Memória** dos programas, baseados no tamanho do software:

- **Model Small (Pequeno):** Usava ponteiros *Near* por padrão para código e dados. O programa inteiro estava limitado a 64 KB de código e 64 KB de dados. Era extremamente rápido.
- **Model Compact (Compacto):** Usava ponteiros *Near* para código (até 64 KB de funções), mas ponteiros *Far* para dados (permitindo arrays e variáveis maiores que 64 KB).
- **Model Large (Grande):** Usava ponteiros *Far* para código e dados por padrão. O programa podia ter tamanho máximo, mas rodava de forma mais lenta devido à constante troca de segmentos.

1. Como os Ponteiros Far aparecem no Assembly (LDS e LES)

Quando trabalhamos com ponteiros **Near**, usamos instruções comuns como **MOV AX, [BX]**, porque o segmento padrão (**DS**) já é assumido.

Porém, para carregar um ponteiro **Far** de 32 bits (4 bytes) da memória para os registradores da CPU, precisamos atualizar **o registrador de segmento e o registrador de deslocamento ao mesmo tempo**. É para isso que servem as instruções **LDS** (Load Data Segment) e **LES** (Load Extra Segment).

O Cenário no Código:

Imagine que você tem uma variável na memória chamada **ponteiro_remoto** que guarda um ponteiro **Far** (4 bytes). Os 2 primeiros bytes guardam o *Offset* e os 2 bytes finais guardam o *Segmento*.

```
DATA SEGMENT
    ; Define um ponteiro Far que aponta para o endereço físico B800:0010
    ; (Antiga memória de vídeo do DOS)
    ponteiro_remoto DD 0B80000010H ; 'DD' significa Data Double-word (4 bytes)
DATA ENDS

CODE SEGMENT
START:
    ; Configura o segmento de dados padrão
    MOV AX, DATA
    MOV DS, AX
```

```

; ---- USANDO LDS ----
; Esta instrução faz duas coisas magicamente:
; 1. Copia o Offset (0010H) para o registrador SI
; 2. Copia o Segmento (B800H) para o registrador DS
LDS SI, [ponteiro_remoto]

; ---- USANDO LES ----
; Faz exatamente a mesma coisa, mas preserva o DS e joga o segmento no ES:
; 1. Copia o Offset (0010H) para o registrador DI
; 2. Copia o Segmento (B800H) para o registrador ES
LES DI, [ponteiro_remoto]

; Agora você pode ler ou escrever no endereço distante usando o ES:
MOV AL, 'A'
MOV ES:[DI], AL    ; Escreve o caractere 'A' diretamente na memória de vídeo
CODE ENDS

```

2. O que acontece com a Pilha (Stack) nas chamadas `CALL`

A pilha é usada para lembrar o caminho de volta quando uma função termina (instrução `RET`). O comportamento muda drasticamente se a função estiver perto ou longe.

Chamada Near (`CALL near / RETN`)

Usada quando a função que você está chamando está **dentro do mesmo segmento de código (CS)**.

1. **Ação do `CALL`:** A CPU empilha **apenas o `IP`** (Instruction Pointer) atual (2 bytes).
2. **O Pulo:** O `CS` continua igual. O `IP` é atualizado com o endereço da função.
3. **Ação do `RET`:** Desempilha os 2 bytes de volta no `IP`.

Estado da Pilha na Função Near:

Topo da Pilha -> [`IP` de Retorno (2 bytes)]

Chamada Far (`CALL far / RETF`)

Usada quando a função está em **outro segmento de código**, muito comum ao chamar funções do próprio sistema operacional (MS-DOS/BIOS) ou de bibliotecas grandes.

1. **Ação do `CALL`:** A CPU precisa salvar o caminho completo. Ela empilha primeiro o `CS` **atual** (2 bytes) e depois o `IP atual` (2 bytes). Totalizando 4 bytes na pilha.
2. **O Pulo:** A CPU carrega o novo `CS` e o novo `IP` da função alvo.
3. **Ação do `RET`:** A instrução de retorno precisa ser um `RETF` (Retorno Far). Ela desempilha os 2 bytes para o `IP` e depois mais 2 bytes para o `CS`.

Estado da Pilha na Função Far:

Topo da Pilha -> [IP de Retorno (2 bytes)]
[CS de Retorno (2 bytes)]

Por que isso causava bugs terríveis?

Se um programador errasse o tipo de retorno, o sistema travava na hora.

Se você chamasse uma função via **CALL far** (colocando 4 bytes na pilha), mas a função terminasse com um **RET** comum (Near, que remove apenas 2 bytes), o processador extraía o antigo **CS** da pilha achando que era um endereço de código comum, pulando para uma região completamente aleatória da memória e **causando o congelamento da máquina**.